

Assignment 3 (20 points)

For Questions 1–6, implement the required expressions or functions in separate C functions. Write a small main program to test each function. Include your source code and the corresponding program output in your submission.

The coding restrictions apply only to the required function implementations, not to the main program used for testing.

For these problems, we will artificially restrict what programming constructs you can use to help you gain a better understanding of the bit-level, logic, and arithmetic operations of C. In answering these problems, your code must follow these rules:

Assumptions

- Integers are represented in two's-complement form.
- Right shifts of signed data are performed arithmetically.
- Data type `int` is `w` bits long. For some of the problems, you will be given a specific value for `w`, but otherwise your code should work as long as `w` is a multiple of 8. You can use the expression `sizeof(int) << 3` to compute `w`.

Forbidden

- Conditionals (`if` or `?:`), loops, switch statements, function calls, and macro invocations.
- Division, modulus, and multiplication.
- Relative comparison operators (`<`, `>`, `<=`, and `>=`)

Note: The coding restrictions apply only to Problems 1–6.

Allowed operations

- All bitwise operators (`&`, `|`, `^`, `~`) and logical operators (`!`, `&&`, `||`).
- Left and right shifts, but only with shift amounts between 0 and `w-1`.
- Addition and subtraction.
- Equality (`==`) and inequality (`!=`) tests. (Some of the problems do not allow these.)
- Integer constants `INT_MIN` and `INT_MAX`.
- Casting between data types `int` and `unsigned`, either explicitly or implicitly.

Even with these rules, you should try to make your code readable by choosing descriptive variable names and using comments to describe the logic behind your solutions.

1. Write a C expression that will yield a word consisting of the least significant byte of `x`, and the remaining bytes of `y`. For operands `x = 0x89ABCDEF` and `y = 0x76543210`, this would give `0x765432EF`. (1 point)

2. Write C expressions that evaluate to 1 when the following conditions are true, and to 0 when they are false. Assume x is of type int.
- Any bit of x equals 1.
 - Any bit of x equals 0.
 - Any bit in the least significant byte of x equals 1.
 - Any bit in the most significant byte of x equals 0.

Your code should follow the integer coding rules above, with the additional restriction that you may not use equality (==) or inequality (!=) tests.

You should write a separate expression for each condition. Each condition is evaluated independently. (2 points)

3. Write code to implement the following function:

```
/* Return 1 when x contains an odd number of 1s; 0 otherwise. Assume w=32. */
int odd_ones(unsigned x);
```

Your function should follow the above bit-level integer coding rules, except that you may assume that the data type int is 32 bits wide.

Your code should contain a total of at most 12 arithmetic, bit-wise, and logical operations. (2 points)

4. Write code to implement the following function:

```
/* Generate mask indicating leftmost 1 in x. Assume w=32.
 * For example 0xFF00 -> 0x8000, and 0x6600 -> 0x4000.
 * If x = 0, then return 0.
 */
int leftmost_one(unsigned x);
```

Your function should follow the above bit-level integer coding rules, except that you may assume that data type int has w=32 bits. Your code should contain a total of at most 15 arithmetic, bit-wise, and logical operations. (2 points)

5. Write code for a function with the following prototype:

```
/* Perform a left rotation . Assume w = 32 and 0 <= n < w.
 * Examples when x = 0x12345678 and w = 32:
 * n=4 -> 0x23456781, n=20 -> 0x67812345
 */
```

```
unsigned rotate_left(unsigned x, int n);
```

Your function should follow the bit-level integer coding rules. Reminder: shifting by w is undefined in C; handle n=0 without doing x >> (w-n). (2 points)

6. Write code for a function with the following prototype:

```
/* Divide by power of two. Assume  $0 \leq k < w - 1$  */
```

```
int divide_power2(int x, int k);
```

The function should compute $x / 2^k$, rounding toward zero, as required by C integer division. (2 points)

7. Consider a **6-bit** two's complement representation. Fill in the empty boxes in the following table. For this question, assume all arithmetic results are truncated to 6 bits using modulo 64 arithmetic. Interpret the resulting bit pattern as a signed two's complement number. (2 points)

Number	Decimal Representation	Binary Representation
Zero	0	
n/a	-1	
n/a	5	
n/a	-10	
n/a		01 1010
n/a		10 0110
TMax		
TMin		
TMax+1		
TMax-1		
TMin+1		
TMin-1		
-TMax		
-TMin		

8. Convert the following decimal numbers to 12-bit unsigned binary numbers. Add leading zeros if necessary. (1 point)

- a. 579
- b. 448
- c. 312

9. Convert the following **two's complement** 12-bit binary numbers to decimal numbers. (1 point)

- a. 110101011101
- b. 000101010101
- c. 101010101010

10. Convert the following decimal numbers to **12-bit two's complement** binary numbers. (1 point)

- a. -623
- b. 445
- c. -112

11. Complete the following 8-bit two's complement binary calculations. Show the full result and the two's complement 8-bit truncated result. Use 9-bit two's complement for the full addition result and 16-bit two's complement for the full multiplication result. For the truncated results, retain only the least significant 8 bits and interpret them as signed two's complement integers. (3 points)

a) $x = 11100111$, $y = 10100111$

$x + y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x + y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____

b) $x = 01100011$, $y = 01100101$

$x + y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x + y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____

c) $x = 00010111$, $y = 00001010$

$x + y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x + y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Full)	Binary: _____	Decimal: _____
$x * y \Rightarrow$ (Truncated 8-bit)	Binary: _____	Decimal: _____

12. We are given the task of generating code to multiply signed integer variable x by different constant factors K . To be efficient, we want to use only the operations $+$, $-$, and $\ll$. For the following values of K , write C expressions to perform the multiplication using at most three operations per expression. Assume x is nonnegative and all intermediate and final results are representable as `int`. (1 point)

- A. $K = 17$:
- B. $K = -7$:
- C. $K = 60$:
- D. $K = -112$: